

GY-53 + ESP32

El GY-53 trae microcontrolador propio y arranca en **modo serie**: te manda la distancia en milímetros, ya calculada, sin librería. Esa es la forma recomendada y la que cubre esta guía primero. El modo I²C queda como alternativa.

Empezá por acá

Modo serie · PS sin conectar · sin librería

`gy53_uart_esp32.ino`

Alternativa

Modo I²C · PS a GND · necesita librería

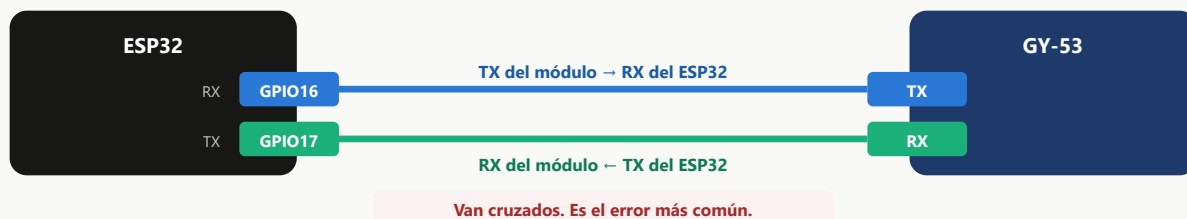
`v15310x_i2c_esp32.ino`

Usá el modo I²C sólo si tu proyecto ya tiene un bus I²C armado y no querés gastar un puerto serie, o si necesitás control fino del sensor (perfiles, calibración de cross-talk).

1 Modo serie — `gy53_uart_esp32.ino`

Conexión al ESP32 clásico

GY-53	ESP32	NOTA
VCC (pin 1 o 12)	3V3 o 5V	El módulo acepta 3–5 V (manual V1.0)
GND (pin 2 o 11)	GND	—
TX (pin 3)	GPIO16	TX del módulo va al RX del ESP32
RX (pin 4)	GPIO17	RX del módulo va al TX del ESP32
PS (pin 6)	sin conectar	Queda en alto = modo serie. No lo toques.
PWM (pin 5)	—	Opcional, si querés leerlo sin puerto serie



En el **ESP32-C3 SuperMini** cambiá `PIN_RX` y `PIN_TX` por pines libres (por ejemplo 3 y 4): el C3 tampoco tiene pines de UART fijos.

LIBRERÍA NECESARIA

Ninguna

El STM32 del módulo hace las cuentas. El programa sólo arma la trama de 8 bytes y verifica el checksum.

Pasos

- 0 **Sacá la película protectora** que viene pegada sobre la ventana del sensor. Es transparente y fácil de no ver; con ella puesta no mide.
- 1 Instalá el soporte de placas ESP32 si no lo tenés: `Archivo` → `Preferencias` → `URLs adicionales` → `https://espressif.github.io/arduino-esp32/package_esp32_index.json`, después `Gestor de tarjetas` → `esp32` → instalar.

- 2

Cableá según la tabla, con PS **sin conectar**.
- 3

Abre `gy53_uart_esp32.ino` y subilo.
- 4

Abri el Monitor Serie a **115200**. (El módulo habla a 9600, pero el monitor va a 115200: son dos puertos distintos.)

```
GY-53 en modo serie. Esperando datos...
1077 mm (107.7 cm) modo=alta precision
1078 mm (107.8 cm) modo=alta precision
```

Cambiar el modo de medición

En `setup()` hay cuatro líneas, una por modo. Descomentá la que quieras:

LÍNEA	RANGO	TIEMPO	PRECISIÓN
<code>enviarComando(0x50)</code>	0 – 2 m	35 ms	±4 cm
<code>enviarComando(0x51)</code>	0 – 1,2 m	22 ms	±3 cm
<code>enviarComando(0x52)</code>	0 – 1,2 m	200 ms	±1 cm — de fábrica
<code>enviarComando(0x53)</code>	0 – 1,2 m	35 ms	±2 cm

!

Guardá la configuración o se pierde al apagar

Los comandos de modo y de baudios son volátiles. Para fijarlos: descomentá `enviarComando(0x25);` en `setup()`, subí **una sola vez**, y volvé a comentarlo. Si lo dejás descomentado, escribís la memoria del módulo en cada arranque sin necesidad.

i Para usar I²C hay que poner PS a GND

Con PS a masa, el STM32 del módulo se desentiende y le hablás al chip VL53L0X directamente. **En este modo el PWM deja de funcionar** y necesitás librería.

Conexión

ESP32 CLÁSICO (DEVKIT CH340C)		ESP32-C3 SUPERMINI	
VCC	3V3	VCC	3V3
GND	GND	GND	G
SDA (pin 9)	GPIO21	SDA (pin 9)	GPIO5
SCL (pin 10)	GPIO22	SCL (pin 10)	GPIO6
PS (pin 6)	a GND	PS (pin 6)	a GND
		No uses GPIO8 y GPIO9: tienen el LED y el botón BOOT.	

Librería

VL53L0X DE POLOLU
Arduino IDE: Programa → Incluir librería → Administrar bibliotecas..., buscá **VL53L0X** y elegí la de **Pololu**.
PlatformIO: lib_deps = pololu/VL53L0X
Si preferís la de Adafruit (Adafruit_VL53L0X) también sirve, pero la API es distinta y este código no compila con ella.

Ajustar el modo

LÍNEA EN	QUÉ HACE
setMeasurementTimingBudget(33000)	33 ms — por defecto
setMeasurementTimingBudget(200000)	200 ms — alta precisión (±3 %)
setMeasurementTimingBudget(20000)	20 ms — rápido (±5 %)

3 Errores comunes

SÍNTOMA	CAUSA	SOLUCIÓN
Serie muda (modo UART)	TX y RX no cruzados	TX del módulo al RX del ESP32. Es el error más común
	PS conectado a GND	Para modo serie, PS va sin conectar
	El módulo está a 115200	Cambiá BAUDIOS en el programa y probá
Checksum invalido seguido	Baudios mal	Probá 115200 antes de dar el módulo por fallado
No se detecta el VL53L0X (modo I²C)	PS quedó al aire	Para I²C, PS va a GND
	SDA y SCL cruzados	Dalos vuelta
	Pines I²C mal en el C3	Usá 5 y 6, no 8 y 9
El escáner muestra 0x29 y no 0x52	No es un error	0x52 es de 8 bits, 0x29 de 7. Mismo sensor
Fuera de rango siempre, módulo nuevo	Película protectora puesta	Sacala de la ventana del sensor
Fuera de rango siempre	Nada que reflejar, u objeto muy lejos	Probá con una hoja blanca a 20 cm
Fuera de rango al aire libre	Luz solar. El sol satura el sensor a 940 nm	Es física, no software. En exterior el alcance cae a 40–80 cm
Lecturas erráticas	Objetivo oscuro, transparente o en ángulo	Vidrio, espejos, metal pulido y superficies inclinadas devuelven mal el pulso

SÍNTOMA	CAUSA	SOLUCIÓN
Valores erráticos	Fuente floja: pide picos de 40 mA	Cables cortos y buena fuente
Mide menos de lo esperado	Objetivo gris u oscuro	Un objetivo al 17 % da 80 cm donde uno blanco da 2 m
No mide detrás de un vidrio	Crosstalk óptico	Requiere calibración de cross-talk (Rev 2, sección 2.3.3)
Se desconfigura al apagar	No guardaste	Mandá <code>0xA5 0x25 0xCA</code> una vez
Falla en frío o mucho calor	Fuera del rango térmico	El chip es de -20 a +70 °C

4 Varios sensores a la vez

EN MODO SERIE — ES FÁCIL

Cada módulo va a un UART distinto del ESP32. El ESP32 clásico tiene **tres** puertos serie por hardware, así que podés colgar dos sensores sin hacer nada raro (uno queda para el USB).

EN MODO I²C — HAY QUE TRABAJAR

Todos arrancan en `0x29`. Hay que conectar el XSHUT de cada uno a un GPIO distinto, arrancarlos apagados, y encender de a uno cambiándole la dirección con `sensor.setAddress()`.